MINISA: Minimal Instruction Set Architecture for Next-gen Reconfigurable Inference Accelerator

Abstract— **Modern reconfigurable AI accelerators rely on rich mapping and data-layout control to sustain high utilization across matrix multiplication, convolution, and emerging non-AI domains. However, exposing this flexibility through fine-grained micro-control results in large instruction footprints and non-trivial on-/off-chip control overheads, limiting effective tile sizes and increasing instruction-fetch traffic. This paper presents MINISA, a minimal instruction set that programs a FEATHER accelerator at the granularity of Virtual Neurons (VNs)—the smallest hardware dot-product atom—while preserving the legal mapping and layout space supported by the underlying architecture. MINISA factorizes control into three layout-setting instructions for input, weight, and output VNs and a single mapping instruction for compute tiles. We further introduce FEATHER+, a modest architectural refinement that eliminates redundant on-chip data replication required by mixed dataflow/layout execution, supporting dynamic inputs and weights. Across diverse workloads, MINISA reduces the control footprint and instruction fetch traffic, freeing on-chip storage for larger compute tiles and improving end-to-end performance without sacrificing built-in hardware flexibility. Our results show that a VN-level abstraction is the coarsest control granularity that retains inter-PE mapping flexibility and the finest granularity that avoids unnecessary control costs. Across 58 GEMM workloads spanning AI (GPT-oss), FHE, and ZKP, MINISA reduces geometric mean off-chip instruction traffic by factors ranging from $35\times$ to $(4.45 \times 10^5)\times$ under various sizes. This eliminates instruction-fetch stalls that consume $46\% - 98\%$ of micro-instruction cycles, yielding up to $99.4\times$ end-to-end speedup for 128×128 FEATHER+.**

I. Introduction

Reconfigurable AI accelerators increasingly serve as general-purpose engines for structured linear algebra beyond conventional deep learning. Their performance stems from two sources: (i) high-throughput low-precision matrix/vector execution and (ii) programmable data movement that enables diverse dataflows and layouts. This programmability is now crucial not only for CNNs and Transformers, but also for workloads that can be compiled into matrix and vector operators, including homomorphic encryption (HE) and zero-knowledge proofs (ZKP). Yet this same flexibility creates a growing challenge: **control overhead**.

FEATHER [29] represents the SotA accelerator enabling (dataflow, layout) coswitching for each individual workload. However, its programming interfaces expose **fine-grained** or **micro-coded** control over buffer layouts, on-chip routing, and PE-to-data mappings. While expressive, such approaches inflate instruction traces, increase off-chip instruction fetch traffic, and consume valuable on-chip instruction memory. The resulting control footprint can directly limit the size of compute tiles that fit on the chip, reducing arithmetic intensity and hurting end-to-end throughput. As accelerators scale in array dimensions and workload diversity, this control-pressure problem becomes a first-order bottleneck (§III-C).

This paper argues that the right abstraction boundary for minimizing control overhead is the **Virtual Neuron (VN)**. A VN corresponds to the smallest hardware dot-product atom in the PE array. A **coarser-grained** abstraction than current VN would ignore inter-PE mapping flexibility, while a **finer-grained** abstraction would introduce unnecessary control costs without gaining flexibility. Therefore, programming at the level of VN is exactly the level to minimize control overhead without losing hardware built-in flexibility.

Building on this insight, we present a compact VN-abstracted instruction set architecture, MINISA, that captures the flexibility in mapping and layout of FEATHER [29], one SotA reconfigurable accelerator, using only four instructions: `SetIVNLayout`, `SetWVNLayout`, `SetOVNLayout`, and `SetMapping`. Layout-setting instructions parameterize the nested-loop order of VNs in streaming and stationary buffers. The mapping instruction triggers execution of a compute tile under the current layout configuration. For a single layer, the program trace consists of three layout instructions followed by a sequence of SetMapping invocations. For consecutive layers, the output of the first layer becomes the input of the next, allowing SetIVNLayout to be amortized across layers.

While MINISA compresses FEATHER's control footprint, the baseline FEATHER microarchitecture still creates two practical frictions for dynamic workloads: it can induce redundant on-chip data duplication, and its buffer/interconnect semantics can implicitly assume that one operand (often weights) is known early enough to be organized in a preferred stationary form. This assumption limits applicability when both operands may arrive or change at runtime, as in dynamic or multi-tenant settings. To remove these barriers without redesigning the accelerator, we propose FEATHER+, a minimal refinement that clarifies the responsibilities between interconnect and buffers and eliminates duplication paths, enabling efficient runtime dataflow/layout reconfiguration for inputs and weights that are not known ahead of time. Our contributions are:

- We identify control overhead as a key limiter of tile size and instruction efficiency in reconfigurable AI accelerators, instruction fetching stall share rises from 0% (smaller than 8×8 FEATHER) to 98% (128×128 FEATHER).
- We propose FEATHER+, a modest enhancement that (1) supports dynamic input data and (2) removes redundant on-chip data duplication when switching dataflows.
- We introduce MINISA, a minimal VN-level ISA for FEATHER that preserves hardware-supported mapping and

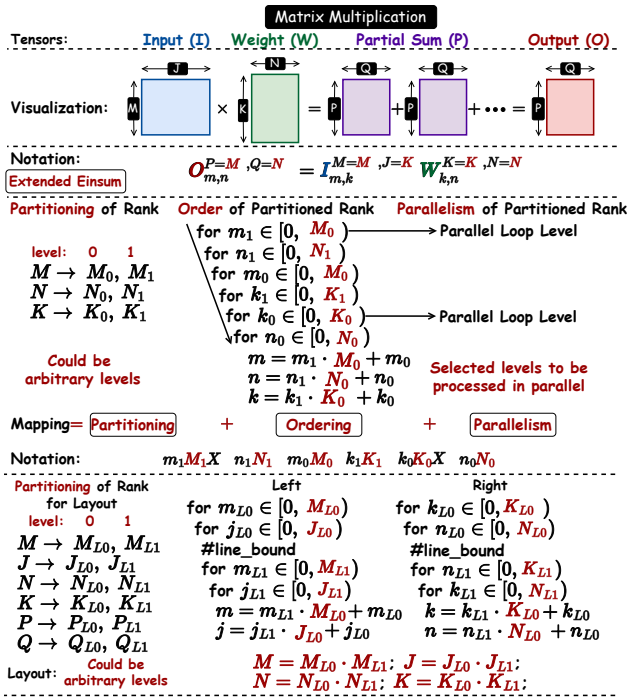

Fig. 1: Mapping and layout illustration for matrix multiplication. We use color coding in this paper: blue/green/purple/red for Input(I)/Weight(W)/Partial-sum(psum, P)/Outputs(O).

layout flexibility with four instructions.

• We demonstrate that MINISA enables up-to $99.4\times$ speedups by avoiding instruction-fetch stalls. We show MINISA reduces instruction-to-data ratio from $58\times$ (micro-instruction programming) to 1.3×10^{-4} (MINISA) for 128×128 FEATHER+. MINISA maintains $< 0.1\%$ instruction-cycle fraction.

II. Background

A. Workload Specification

Modern AI inference workloads—from LLM attention blocks to text-to-image pipelines—are dominated by matrix multiplications and convolutions whose *operand values and shapes* can vary across layers and requests. To describe this diversity in a uniform way, we adopt *extended einsum* notation [8]. Taking matrix multiplication as an example (Fig. 1),

$$O_{m,n}^{P=M,Q=N} = I_{m,k}^{M=M,J=K} \cdot W_{k,n}^{K=K,N=N},$$

each tensor is annotated with:

• **Superscript**: an ordered list of *rank names* that define the logical shape (e.g., $[M, J]$ for $Left$). Ranks are capitalized.

• **Subscript**: an ordered list of quasi-affine *index expressions* over rank variables and constants (e.g., $[m, k]$). Rank variables use the lowercase form of the corresponding rank name.

$$I \in \mathbb{R}^{M\times J}, W \in \mathbb{R}^{K\times N}, P \in \mathbb{R}^{P\times Q}, O \in \mathbb{R}^{P\times Q}.$$

Partial sums represent intermediate accumulation states that arise when the rank variable of reduction rank (e.g., k) is distributed across non-consecutive loop levels in mapping, requiring aggregation of partial results to produce final outputs.

B. Mapping and Layout

A given extended einsum can be executed with many legal schedules. We refer to a fine-grained schedule of compute and memory accesses as a **mapping** (or dataflow), which specifies (i) rank partitioning, (ii) ordering of partitioned ranks, (iii) parallelization choices, and (iv) shape parameters[1] (Fig. 1).

Independently, each tensor can be organized differently in on-chip memories. We use **layout** to denote the fine-grained placement order of data in the on-chip buffers [27], [29]. Each logical rank is partitioned into **layout partitioned ranks**; a layout then specifies the loop order of these ranks. Ranks above `#line_bound` determine the order across memory lines, while ranks below determine the order within a line (Fig. 1). This separation is essential for reconfigurable accelerators that must co-switch compute scheduling and buffer organizations across workloads.

C. Reconfigurable Accelerator

No single dataflow is globally optimal across the wide shape space of modern operators [21], [27]. This motivates **reconfigurable accelerators** that can adapt mappings and layouts at runtime to sustain high utilization across workloads [4], [15]–[18], [22], [29], [31]. FEATHER [29] is a recent practical design that enables low-cost *co-switching* of dataflow and layout, making reconfigurable execution deployable.

However, FEATHER has two limitations.

• **Point-to-point connection from buffers to computation array**. Such rigid connections require weights to be preknown and offline reordered into desired layout. It works for general convolution but is inefficient for recent LLMs when both inputs and weights can arrive or change at runtime. Further, the offline reorganized weights might require *redundant on-chip data replication*.

• **Heavy Control Overhead**. Programming individual switches require significant configurations overhead, which even dominates at large scale.

To eliminate both, we augment it into **FEATHER+** with two changes in §III, and introduce the Virtual Neuron abstraction and VN-based ISA set, MINISA, for FEATHER+ in §IV.

III. FEATHER+ and Programming View

This section introduces FEATHER+ and programming view. And it analyzes why Virtual Neuron is the exact level of programming abstraction for FEATHER+.

A. Architectural Refinement of FEATHER into FEATHER+

To avoid the limitation of point-to-point connection from buffers to computation array, we introduce **FEATHER+** (Fig. 2), a modest architectural refinement with two changes:

• **All-to-all distribution from buffers to compute array**. We replace the per-column point-to-point connection from the streaming and stationary buffers to NEST with two separate all-to-all crossbars. This allows any resident VN to be flexibly multicast to arbitrary PE columns, eliminating the need to

[1]Mapping is a dataflow (partitioning + ordering + parallelism) with shape information [14]. We use *mapping* and *dataflow* interchangeably.

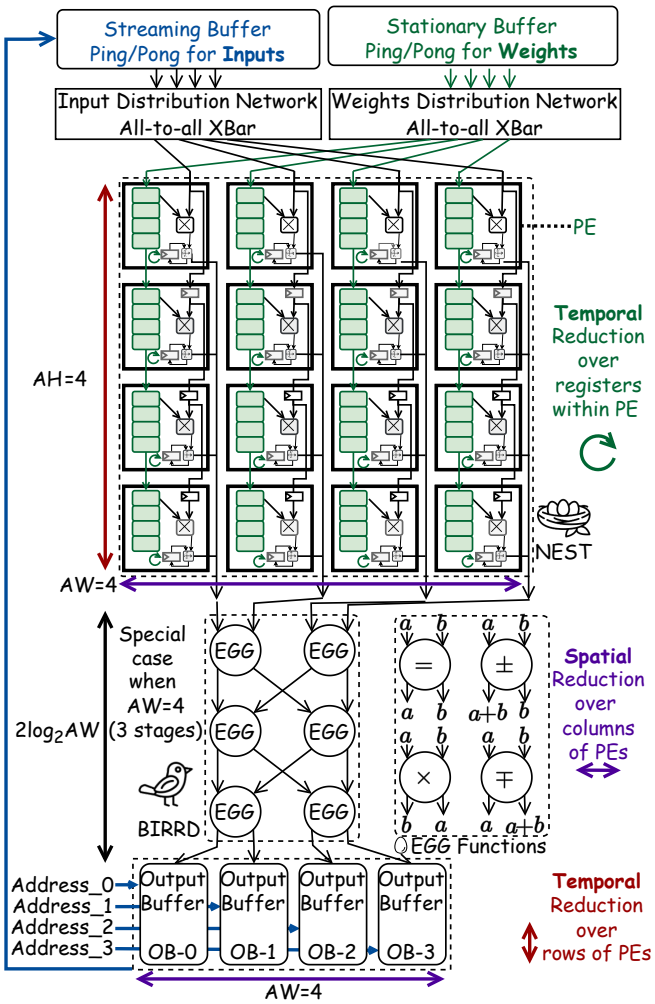

Fig. 2: Programmer view of FEATHER+, the reconfigurable accelerator capable of co-switching both dataflow and layout.

explicitly materialize duplicated copies of the same data in on-chip buffers. Such crossbar-style distribution is common in AI ASICs [9]–[11], as it is resource-efficient.

- **Simplified streaming-buffer banking**. With all-to-all distribution, FEATHER+ no longer requires a multi-bank streaming-buffer interface to satisfy per-column address constraints. We therefore simplify the streaming buffer into a single logical bank with a cleaner VN-level access model.

Together, these changes (i) remove the requirement that one operand must be known ahead of execution to be placed in a preferred stationary form, and (ii) enable duplication-free data movement across on-chip buffers. FEATHER+ serves as the default hardware backend for this work.

B. Programming View of FEATHER+

From a programmer's perspective, FEATHER+ comprises an $AH \times AW$ PE array (**NEST**) and a multi-stage additive interconnection network (**BIRRD**) (Fig. 2). **Each PE holds AH local registers, because this ensures each row of PEs to generate results in the round-robin manner**. The architecture gives three-level reduction:

- temporal reduction within a PE, performing AH-way dot product to generate a single partial sum (psum).
- spatial reduction over psums from a row of PEs via BIRRD.
- temporal reduction of psums from a column of PEs in OB.

Further, for each column of PE, inputs move from top to bottom in the pipelined manner. To ensure correct accumulation, weights in the same column must correspond to the same reduction index (e.g. value of k in Fig. 1 for matrix multiplication). This structure enables flexible reduction patterns under the constraint, positioning FEATHER+ as a *constrained dataflow accelerator* [26].

C. Control Overhead of Reconfigurable Accelerator

While FEATHER+ exposes broad mapping/layout flexibility, *realizing* this flexibility via switch- and cycle-level micro-control can incur substantial overheads. Programs must frequently specify interconnect control, PE-to-data selection, and buffer/accumulation behaviors. This has two direct consequences. First, control state consumes on-chip storage that could otherwise be used for activations and weights, limiting the largest feasible tile size and reducing arithmetic intensity. Second, fetching long instruction traces increases off-chip instruction traffic, which is especially harmful for memory-bound layers. As accelerator arrays scale and workload diversity expands, this control pressure becomes a first-order limiter of end-to-end throughput. When adopting micro-control to configure each individual switches for FEATHER, significant memory overhead of instructions fetching would dominate at larger scale, as shown in Tab. I.

TABLE I: Explicit compute stall of fetching instructions

FEATHER	4×4	8×8	16×16	32×32	64×64	128×128
stall	0	0	46.2%	84.5%	95.3%	98%

D. Insight: Dot Products (VN) is the Right Abstraction

MINISA is based on a simple architectural observation: FEATHER+ always computes at the granularity of a PE dot product. We therefore define a **Virtual Neuron (VN)**[2] as the smallest software operand fragment that matches this hardware atom, i.e. AH-element dot product. This leads to a key insight:

Programming at VN granularity is the coarsest control that still preserves inter-PE mapping flexibility and the finest control that avoids unnecessary switch-level overhead.

Concretely, MINISA re-expresses FEATHER+'s legal mapping and layout space using VN-level layout descriptors for input, weight, and output operands and a single VN-level mapping instruction that triggers execution of a compute tile. This factorization retains FEATHER+'s built-in reconfigurability while dramatically compressing control footprint and instruction-fetch demands, enabling larger on-chip tiles and higher end-to-end performance.

IV. MINISA

This section presents MINISA, a minimal instruction set that exposes FEATHER+'s reconfigurability at the *Virtual Neuron* (VN) granularity. We first define operand-specific VNs and the

[2]we borrow this term from MAERI [16] which was a flexible dataflow accelerator that defined VN to be a flexible sized dot product engine.

FEATHER+ mapping constraints, then introduce VN-layout representations for operand-specific VNs (I_{VN}, W_{VN}, O_{VN}).

A. Overview of MINISA

MINISA is built on a simple observation: FEATHER+'s minimal compute unit is a PE that performs an AH-way dot product. Rather than configuring switch-level micro-operations, MINISA treats this AH-element dot product as the atomic unit of control. This abstraction preserves FEATHER+'s built-in flexibility while compressing the configuration space into a small set of architectural parameters and compact layout descriptors.

1) Virtual Neuron Abstraction:

a) Definition of VN: A *Virtual Neuron* (VN) VN is the minimal hardware dot-product atom, which is also the largest unit we can treat as atomic in software without paying per-wire/per-cycle control. For an $AH \times AW$ FEATHER+ NEST, each PE performs an AH-way dot product, hence the VN size is AH.

b) Operand-specific VNs: We explicitly distinguish: I_{VN}, W_{VN}, $P_{VN}$and O_{VN}. Each denotes the smallest operand fragment **consumed at one PE dot product** and **produced by a column of PE's dot products** in NEST.

B. FEATHER+ Mapping Flexibility and Constraints

FEATHER+ couples compute and reduction across temporal and spatial dimensions. The architecture enforces a key constraint that shapes the MINISA mapping space.

a) Reduction-aware mapping: Within a PE column, inputs stream top-to-bottom and are dot-producted with stationary weights. Correct local reduction requires all PEs of the column to share the same *weight VN row index*. This property forms the basis of MINISA's constrained parametric mapping.

b) Justification of Minimum: A coarser-grained abstraction than current VN would ignore inter-PE mapping flexibility, while a fine-grained abstraction introduces unnecessary costs without improving flexibility, hence programming at the level of VN would be the exact level to minimize control overhead without losing hardware built-in flexibility.

C. MINISA for an $AH \times AW$ NEST

We consider an $AH \times AW$ PE array. PE (a_h, a_w) denotes the PE at row $a_h \in \{0, \ldots, AH-1\}$ and column $a_w \in \{0, \ldots, AW-1\}$. Weights are organized as weight VNs indexed by $W_{VN}(r, c)$, where r and c denote the W_{VN} row and column indices. Our objective is to map each PE to a W_{VN} subject to two constraints:

- **Temporal local reduction.** All PEs within a PE column must participate in a local reduction over the *same* W_{VN} row index (same r), i.e. $r(a_h, a_w)$ is independent of a_h.
- **width-wise independence and arbitrary replication.** All columns of PEs in NEST are independent. Each PE column may hold W_{VN} with an arbitrary subset of column indices (c). The same W_{VN} may be replicated onto many PE columns to process different inputs in parallel.

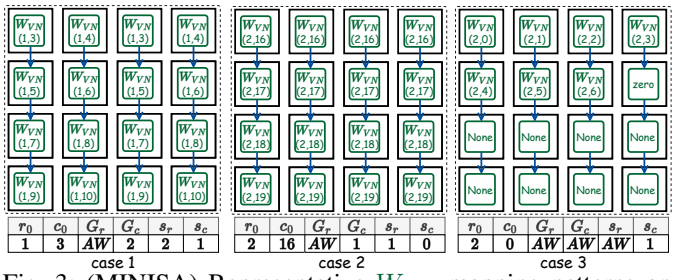

Fig. 3: (MINISA) Representative W_{VN} mapping patterns on a 4×4 NEST under temporal-local-reduction and column-replication constraints.

SetMapping	OpCode	G_r	G_c	r_0, c_0	s_r, s_c
Bitwidth	2	$\lceil \log_2(AH) \rceil$	$\lceil \log_2(AW) \rceil$	$\lceil \log_2(D_{sta}/AH) \rceil$	$\lceil \log_2(D_{sta}/AH) \rceil - 1$
value ranges	11	$[1, AH]$	$[1, AW]$	$0 \le r_0 c_0 \le \lfloor D_{sta} \cdot AH/AW \rfloor$	$[0, D_{sta}/AH/2]$
Constraints: specified loaded W_{VN}				$r_0 < K, c_0 < N$,	$s_r \le K/2, s_c \le N/2$

Fig. 4: MINISA for W_{VN} mapping on an $AH \times AW$ NEST. K, N are shapes of weights matrix.

1) Parametric W_{VN} Mapping: MINISA realizes all legal mappings using a small set (θ) of six parameters:

$$f_\theta : (a_h, a_w) \;\mapsto\; W_{VN}\big(r(a_h, a_w),\, c(a_h, a_w)\big) \tag{1}$$

where $\theta = (r_0, c_0, G_r, G_c, s_r, s_c)$ and

$$r(a_h, a_w) = r_0 + \left\lfloor \frac{a_w}{G_r} \right\rfloor \tag{2}$$

$$c(a_h, a_w) = c_0 + s_r\, a_h + s_c \left(a_w \bmod G_c\right) \tag{3}$$

a) Row index: Eq. (2) assigns W_{VN} row index (r) based solely on a_w. Parameter r_0 selects the base W_{VN} row index, while G_r controls how many PE columns share the same W_{VN} row index. Because r does not depend on a_h, all PEs within a column reduce into a single W_{VN} row index, satisfying the constraint of temporal local reduction.

b) Column index: Eq. (3) decouples temporal (a_h) and spatial movement (a_w) along the W_{VN} column index. Parameter c_0 denotes starting W_{VN} column index. Parameter s_r is the *temporal stride* across PE rows. Parameter G_c defines a replication group of PE columns, and s_c specifies the *spatial stride* among distinct columns within the group. This structure captures column-wise independence and replication.

Any VN with $(r(a_h, a_w), c(a_h, a_w))$ being out of bounds are implicitly zero-padded and not participate in computation.

c) Configuration Bitwidth: Let the W_{VN} matrix have $R_{W_{VN}} = K/AH$ rows and $C_{W_{VN}} = N$ columns. For an $AH \times AW$ PE array, parameter ranges are specified in Fig. 4. If a parameter x takes values in $[0, x_{\max}]$, it needs $\lceil \log_2(x_{\max} + 1) \rceil$ bits.

2) Minimal-bit ISA for VN-Granularity Weight Layouts:

a) SetWVNLayout ISA: It configurates the *stationary buffer* layout for W_{VN}. The stationary buffer stores AW W_{VN}*s per row* with depth D_{sta} (in rows), and each VN must occupy the same slot in AH consecutive rows (see Fig. 5).

TABLE II: Overview of MINISA instructions.

ISA name	ISA meaning
`SetIVNLayout`	Configure the *streaming* buffer layout for input VNs and trigger loading of input data from off-chip memory into the on-chip input buffer.
`SetWVNLayout`	Configure the *stationary* buffer layout for weight VNs and trigger loading of weights from off-chip memory into the on-chip weight buffer.
`SetOVNLayout`	Configure the *output* buffer layout for output VNs and clear (initialize) the on-chip output buffer for accumulation.
`SetMapping`	Specify the VN-level mapping parameters and trigger execution of a single compute tile on the PE array under the current VN layouts.

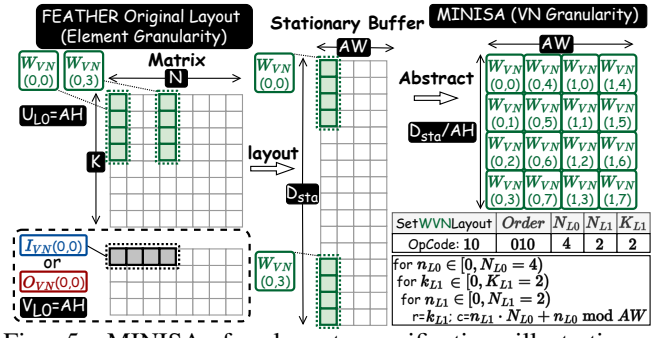

Fig. 5: MINISA for layout specification illustration of SetWVNLayout ($K = 8, N = 8, AH = AW = 4 \Rightarrow K_{L0} = AH$). Takeaway: MINISA organizes layout at VN granularity.

b) Element-wise layout with two-level partitioning: We start from an element-granularity view of the weight matrix of shape (K, N). Each rank is partitioned into two layout levels: $K = K_{L1} \cdot K_{L0}$, $N = N_{L1} \cdot N_{L0}$ with element indices $k = k_{L1} \cdot K_{L0} + k_{L0}$, $n = n_{L1} \cdot N_{L0} + n_{L0}$. The four partitioned ranks $\{k_{L0}, k_{L1}, n_{L0}, n_{L1}\}$ form a general nested-loop layout space that can represent diverse contiguous and strided layout of W_{VN} in the stationary buffer.

c) VN abstraction for weights W_{VN}: The VN abstraction compresses the element-wise layout into a VN-granularity representation. For weights, each VN aggregates AH *consecutive elements along* K for a fixed n. Thus we impose the VN constraint $K_{L0} = AH$, and omit k_{L0} from the nested-loop description. After this constraint, the remaining layout degrees of freedom reduce to the arbitrary ordering of the three ranks

$$\{k_{L1},\ n_{L0},\ n_{L1}\}$$

This constrained loop space is sufficient to capture all VN-respecting stationary-buffer layouts of W_{VN}.

d) VN Indexing: At VN granularity, we identify weight VNs by $W_{VN}(r, c)$, where

$$r = k_{L1}, \qquad c = n_{L1} \cdot N_{L0} + n_{L0}.$$

e) Address of VN in Stationary Buffer: Flattens VNs according to a selected permutation of $\{k_{L1}, n_{L0}, n_{L1}\}$ and then folds the index of flattened VN into the stationary-buffer row and column addresses. Let

$$\mathbf{D} = [K_{L1},\ N_{L0},\ N_{L1}], \qquad \mathbf{I} = [k_{L1},\ n_{L0},\ n_{L1}].$$

Let $P = (p_0, p_1, p_2)$ be a permutation of $\{0, 1, 2\}$ representing the *outer* $\rightarrow$ *inner* loop order. The linear VN index is

$$L = I_{p_0} \cdot D_{p_1} D_{p_2} + I_{p_1} \cdot D_{p_2} + I_{p_2}$$

We map L into stationary-buffer coordinates by

$$\texttt{row} = \left\lfloor \frac{L}{AW} \right\rfloor, \qquad \texttt{col} = L \bmod AW.$$

SetWVNLayout	OpCode	Order	N_{L0}	N_{L1}	K_{L1}
Bitwidth	2	3	$\lceil \log_2(AW) \rceil$	$\lceil \log_2(D_{sta})/AH \rceil$	$\lceil \log_2(D_{sta})/AH \rceil$
value ranges	00	$[0, 5]$	$[1, AW]$	$N_{L1}K_{L1} \leq [D_{sta}/N_{L0}]$	

SetIVNLayout	OpCode	Order	M_{L0}	M_{L1}	J_{L1}
Bitwidth	2	3	$\lceil \log_2(AW) \rceil$	$\lceil \log_2(D_{str})/AH \rceil$	$\lceil \log_2(D_{str})/AH \rceil$
value ranges	01	$[0, 5]$	$[1, AW]$	$M_{L1}J_{L1} \leq [D_{sta}/M_{L0}]$	

SetOVNLayout	OpCode	Order	P_{L0}	P_{L1}	Q_{L1}
Bitwidth	2	3	$\lceil \log_2(AW) \rceil$	$\lceil \log_2(D_{str})/AH \rceil$	$\lceil \log_2(D_{str})/AH \rceil$
value ranges	10	$[0, 5]$	$[1, AW]$	$P_{L1}Q_{L1} \leq [D_{sta}/P_{L0}]$	

Fig. 6: MINISA specifications for input/weight/output layout. D_{sta}, D_{str} refers to the depth of stationary, streaming buffer.

f) Configuration Bitwidth: The VN constraint fixes $K_{L0} = AH$, leaving the ordering of $\{k_{L1}, n_{L0}, n_{L1}\}$ as the only combinational layout choice. There are $3! = 6$ legal permutations, requiring $\lceil \log_2 6 \rceil = 3$ bits. We therefore encode the layout using a 3-bit `Order`. Detailed configurations of different order in partitioned ranks is shown in Tab. III. To respect stationary-buffer capacity, we additionally bound the non-reduction tile extent and the combined extent of the other two ranks. In particular, the inner non-reduction factor is constrained by buffer width $1 \leq N_{L0} \leq AW$, and the product of the remaining rank extents must fit within the buffer depth: $K_{L1} \cdot N_{L1} \leq \left\lfloor \frac{D_{\text{sta}}}{N_{L0}} \right\rfloor$. This coupling reflects that k_{L1} and n_{L1} are *jointly* limited by the available stationary-buffer depth under a chosen N_{L0}.

g) Case study: Fig. 5 illustrates loop order $n_{L0} \rightarrow k_{L1} \rightarrow n_{L1}$ i.e. $P = (n_{L0}, k_{L1}, n_{L1})$. Using the rule above,

$$L = n_{L0}(K_{L1}N_{L1}) + k_{L1}N_{L1} + n_{L1} = 4n_{L0} + 2k_{L1} + n_{L1}.$$

Since $AW = 4$, we obtain $\texttt{row} = n_{L0}, \texttt{col} = 2k_{L1} + n_{L1}$. Thus each stationary-buffer row packs four VNs in the order $W_{VN}(0,\ 0), W_{VN}(0,\ 4), W_{VN}(1,\ 0), W_{VN}(1,\ 4)$ for $n_{L0} = 0$, and similarly for $n_{L0} = 1, 2, 3$, matching the VN-granularity layout shown in the figure. This example highlights how SETWVNLAYOUT compresses the original element-granularity FEATHER layout into a compact VN-level encoding while preserving the intended stationary-buffer ordering.

3) SetIVNLayout and SetOVNLayout ISA: The layouts for input and output VNs are specified in MINISA following the same methodology as SETWVNLAYOUT with a single operand-specific change: configure the *streaming buffer*, since they specify independently in the streamed data path. Further, VN constraint is placed on the reduction-aligned dimension of each operand, e.g. for input matrices of shape (M, J) we set $J_{L0} = AH$, and for output matrices of shape (P, Q) we set $Q_{L0} = AH$. This gives SETIVNLAYOUT and SETOVNLAYOUT as shown in Tab. 6.

D. FEATHER+ Configuration Generations

Under MINISA, the detailed configurations of the distribution networks, NEST, BIRRD, and address generation can be

TABLE III: Unified 3-bit permutation encoding across VN layouts. Each column applies the same `order_id` value to the operand-specific three-rank set remaining after the VN constraint ($K_{L0}=AH$, $J_{L0}=AH$, $Q_{L0}=AH$).

order	W_{VN} order	I_{VN} order	O_{VN} order
000	$k_{L1} \to n_{L0} \to n_{L1}$	$m_{L1} \to m_{L0} \to j_{L1}$	$p_{L1} \to p_{L0} \to q_{L1}$
001	$k_{L1} \to n_{L1} \to n_{L0}$	$m_{L1} \to j_{L1} \to m_{L0}$	$p_{L1} \to q_{L1} \to p_{L0}$
010	$n_{L0} \to k_{L1} \to n_{L1}$	$m_{L0} \to m_{L1} \to j_{L1}$	$p_{L0} \to p_{L1} \to q_{L1}$
011	$n_{L0} \to n_{L1} \to k_{L1}$	$m_{L0} \to j_{L1} \to m_{L1}$	$p_{L0} \to q_{L1} \to p_{L1}$
100	$n_{L1} \to k_{L1} \to n_{L0}$	$j_{L1} \to m_{L1} \to m_{L0}$	$q_{L1} \to p_{L1} \to p_{L0}$
101	$n_{L1} \to n_{L0} \to k_{L1}$	$j_{L1} \to m_{L0} \to m_{L1}$	$q_{L1} \to p_{L0} \to p_{L1}$
110/111	reserved	reserved	reserved

derived from the specified layout and mapping.

• **No Conflicts**: When the inputs of all PE columns are drawn from the same reduction group (i.e., all generated P_{VN}s share the same row index), no output conflicts occur.

• **Output Buffer Port Conflicts**: When the inputs of different PE columns come from different reduction groups (i.e., the generated P_{VN}s have different row indices), output buffer write-port contention may arise if multiple P_{VN}s target the same output buffer in the same cycle. To avoid contention, the controller disables the offending PE columns for that cycle, reducing overall PE utilization. Bank conflicts happen for 6.67% of all potential input configurations in 8×8 FEATHER+.

E. Instruction Set Architecture and Execution Model

1) Instruction Set Architecture: MINISA defines four instructions (Tab. II) to reconfigure architecture and buffer.

• `SetIVNLayout`: and `SetWVNLayout`: Configure the on-chip streaming and stationary buffer layouts for Virtual Neurons of Input (I_{VN}) and Weight (W_{VN}), respectively.

• `SetOVNLayout`: Configures the layout for Output Virtual Neurons (O_{VN}) in the output buffer, which becomes the layout of input for next successive operation.

• `SetMapping`: Configures the parametric mapping of W_{VN} to PEs in NEST. It defines parameters θ for indexing, replication, and striding to specify one compute tile.

2) ISA Bitwidth Analysis: Bitwidth of parameters in ISAs are bounded by static architectural parameters, including D_{sta}/D_{str} referring to the depth of stationary/streaming buffer and AW/AH indicate the width/height of NEST. Workload parameters could be configured at any value within the bound of bitwidth, when actual index of elements exceed workload shape, zero are automatically padded.

3) Execution Triggering and Data Movement: A key architectural distinction in MINISA is the decoupling of memory operations from computation triggers:

• **Memory vs. Compute Triggers:** `SetIVNLayout` and `SetWVNLayout` explicitly trigger the loading of data from off-chip memory to on-chip buffers, while `SetOVNLayout` is responsible for initializing output buffer for new output tile, and dumping current content back to streaming buffer when finishing processing current output tile, i.e. Out→Stream. Conversely, `SetMapping` acts exclusively as execution trigger for a single compute tile.

• **Tiled Processing:** Because the total shape of Weight VNs ($R_{W_{VN}}, C_{W_{VN}}$) may exceed the size of a single compute tile, one layer is often processed in sequential tiles. A single layer's trace typically consists of layout instructions followed by a sequence of `SetMapping` instructions.

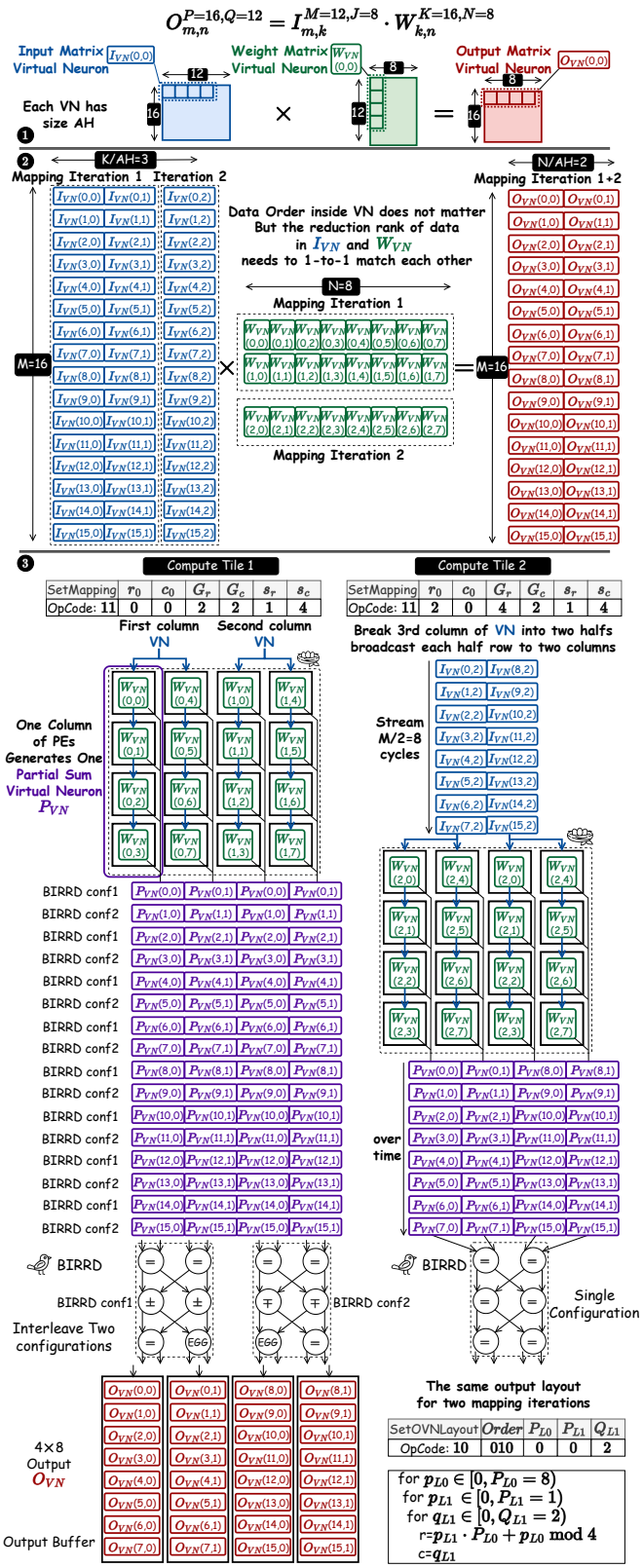

Fig. 7: Mapping a matrix multiplication to FEATHER+ (AH×AW=4×4 NEST) as 2 compute tiles under MINISA.

4) Execution Model:

TABLE IV: Workload specifications of matrix multiplication.

Workload	Matrix Multiplication of Shape $O_{m,n}^{P=M,Q=N} = I_{m,k}^{M=M,J=K} \cdot W_{k,n}^{K=K,N=N}$
FHE: BConv	$(65536\times K)\cdot(K\times N)$, $K\in[28,60]$, $N\in[72,160]$ (41 shapes)
FHE: NTT	$J=K=N\in\{1024,2048,4096\}$, $M\in\{64,128,256\}$, $M\le K/16$
ZKP: NTT	$J=K=N\in\{8192,16384,32768\}$, $M\in\{K/32,K/16\}$
GPT-oss	$M=2048$, $(J=K,N)\in\{(64,2048),(2880,4096/5120/201088),(4096,2880)\}$

a) Single-layer canonical program instruction trace:

$$\texttt{SetI/W/OVNLayout} \rightarrow \{\texttt{SetMapping}\}^{\times T}$$

where T denotes the number of compute tiles required to consume all on-chip input and weight VNs for that layer. The first three instructions establish the operand layouts and perform the necessary data movement and buffer initialization. The subsequent `SetMapping` instructions then sweep the tiles over the loaded data.

b) Multi-layer trace with output reuse: For consecutive layers, the output of layer i becomes the input of layer $i{+}1$. Under this regime, `SetIVNLayout` only needs to be issued once at the beginning of the sequence to define the input VN layout for the first layer. The final output VN layout of layer i is reused as the input VN layout for layer $i{+}1$, avoiding redundant re-specification. Each new layer still issues its own `SetWVNLayout` to load its weights, `SetOVNLayout` to clear and configure its output buffer, and a batch of `SetMapping` instructions to iterate over its compute tiles.

5) Walk-through Case Study: Fig. 7 illustrates this tiling semantics in a concrete walk-through, assuming `SetIVNLayout` and `SetWVNLayout` have already loaded the necessary input and weight VNs onto the chip. In this matrix multiplication (❶), the entire on-chip-resident data (❷) are processed using two compute tiles (❸), realized as two successive `SetMapping` instructions on the same $AH\times AW$ NEST. Each compute tile selects a different subset of the on-chip input and weight VNs while sharing the same `SetOVNLayout`. Both tiles contribute partial sums to the *same* logical O_{VN}, so the O_{VN} layout must remain invariant across the two mapping iterations. This invariant output layout guarantees that the partial results from the first tile and the second tile are accumulated into consistent output coordinates, completing the full computation over the loaded data without requiring additional data movement or re-layout of the outputs.

V. Evaluation

Our evaluation addresses three questions: **(Q1)** How much control overhead does MINISA remove? **(Q2)** What end-to-end performance gains does the MINISA helps FEATHER+ across AI, HE, and ZKP workloads? **(Q3)** What is the resource overhead of FEATHER+ relative to FEATHER?

A. Evaluation Setup

Methodology. We evaluate our design using a dual-level simulation framework: a cycle-accurate RTL model of FEATHER+ (supporting both MINISA and micro-instructions) validates correctness and timing, while a functional parametric analytical model extends our analysis to larger array scales.

TABLE V: Experimental setup for instruction-overhead evaluation (FEATHER+). Bandwidths are in *bytes/cycle* (equivalently GB/s at 1 GHz under our model).

$AW=AH$	On-chip capacity (MB)			Peak	Bandwidth (Byte/cycle)			
	StrB/ StaB	OB	Instr.	MAC/cycle	I	W	O	Instr.
4	1.6	0.8	0.5	16	4	4	16	9
8	6.4	3.2	1.0	64	8	8	32	9
16	25.6	12.8	2.0	256	16	16	64	9
32	102.4	51.2	3.0	1024	32	32	128	9
64	409.6	204.8	5.0	4096	64	64	256	9
128	1638.4	819.2	10.0	16384	128	128	512	9

Architecture Configuration. We model a FEATHER+ array with a square NEST of $AH\times AW$ processing elements (PEs), sweeping dimensions $AH = AW \in \{4,8,16,32,64,128\}$. On-chip data SRAM scales proportional to $AH\cdot AW$ and is partitioned into streaming (40%), stationary (40%), and output (20%) buffers. A dedicated on-chip instruction buffer (scaling from 0.5 MB to 10 MB) is fed via a fixed 9 B/cycle off-chip fetch interface. We model off-chip memory bandwidths of AW B/cycle for inputs/weights and $4AW$ B/cycle for outputs.

Workloads. Our benchmark suite consists of 58 GEMM kernels derived from three domains: LLMs (ChatGPT OSS 20B), Homomorphic Encryption (FHE) bootstrapping [3] and Number Theory Transformation from both Homomorphic Encryption [28] and Zero-Knowledge Proofs (ZKP [19]), as detailed in Tab. IV.

Baselines. For every workload and array configuration, we compare two setups under identical mappings.

- **Micro-instruction (Baseline):** Explicit, fine-grained control where every switch and PE is configured per cycle.
- **MINISA (Ours):** VN-based 9-byte MINISA instructions.

B. Control Overhead Evaluation (Q1)

We first analyze the reduction in control footprint. Fig. 8 compares the total instruction bytes required for MINISA versus the micro-instruction baseline.

Instruction Footprint Collapse. Across all 58 workloads, MINISA reduces instruction bytes by a geometric mean of $1.9\times10^5\times$ for 128×128 FEATHER+. This reduction scales aggressively with array size: while the savings are substantial at small scales ($7.2\times$ at $AW{=}AH{=}4$), they grow exponentially because MINISA's VN-level instructions remain fixed-size, whereas micro-instructions must explicitly address an increasingly large fabric of switches and PEs.

Instruction-to-Data Ratio. The efficiency gap is best illustrated by the ratio of instruction traffic to data traffic. At NEST=128, the micro-instruction baseline requires fetching $49\times$ more instruction bytes than actual matrix data bytes (geomean). This inversion turns control traffic into the dominant bandwidth consumer. In contrast, MINISA's instruction stream is negligible, constituting only 2.6×10^{-4} of the data volume.

Domain-Specific Gains. The reduction is most pronounced in workloads with irregular or frequent control changes. At NEST=128, MINISA reduces instruction bytes by $3.6\times10^6\times$ for ZKP kernels and $1.1\times10^5\times$ for FHE kernels.

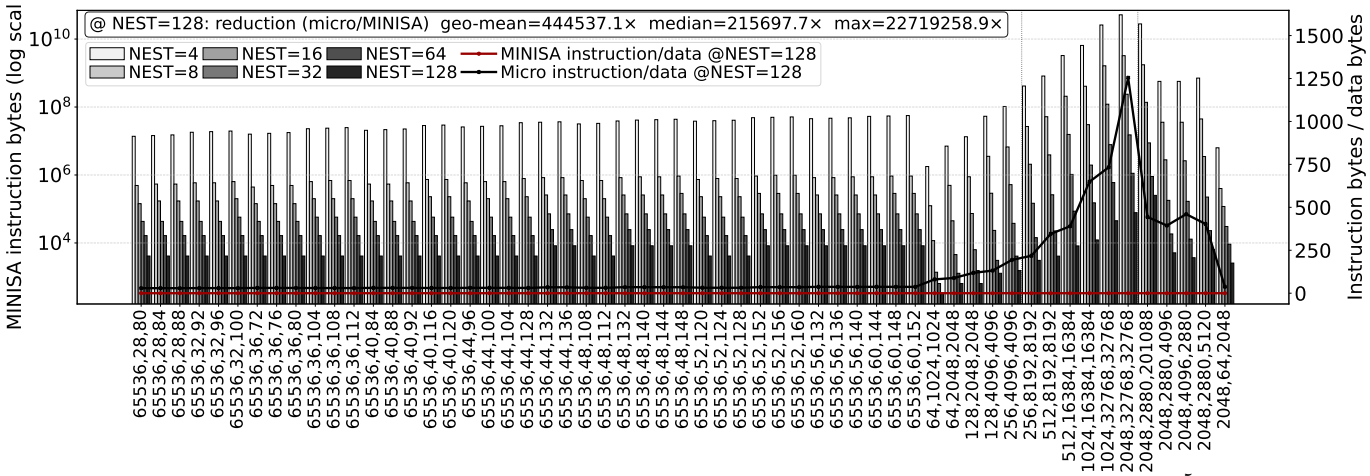

Fig. 8: Instruction overhead reduction. MINISA reduces instruction bytes by a geometric mean of $(2.1\times10^5)\times$ compared to the micro-instruction baseline for 128×128 FEATHER+. Black and red lines illustrate the ratio of instruction size to size of data for baseline and MINISA, separately. **Takeaway**: MINISA reduces the size of instruction by ($\sim35\times$) to $(4.45\times10^5)\times$, effectively removing control overhead from the critical path, making bandwidth-dominating reconfiguration practical.

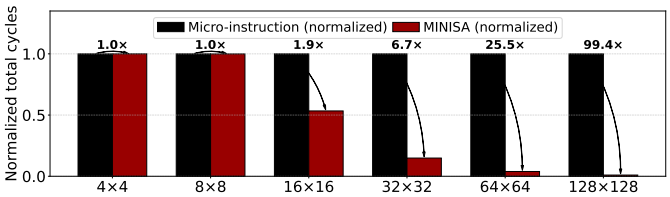

Fig. 9: End-to-end speedup and stall analysis. MINISA achieves up to $99.4\times$ speedup as array size scales to 128×128. **Takeaway:** The performance gain stems from eliminating instruction-fetch stalls. While the micro-instruction baseline becomes instruction-bound (spending up to 98% of cycles stalled) at large scales, MINISA incurs zero instruction-fetch stalls across all configurations.

C. Performance Evaluation (Q2)

1) Performance Evaluation of Programming Abstraction: We next evaluate how control compression translates to end-to-end speedup, shown in Fig. 9.

Eliminating Instruction-Fetch Stalls. Performance gains arise from the removal of instruction-fetch bottlenecks.

- **Small Scale ($<16\times16$):** At 4×4 / 8×8 FEATHER, speedup is effectively $1\times$. Although MINISA reduces the footprint, the baseline's micro-instruction stream is small enough to be hidden behind compute phases.
- **Large Scale ($\geq16\times16$):** As the array grows, the micro-instruction stream saturates the fetch bandwidth. The baseline spends **46.7%** of total cycles stalled on instruction fetch at AW=16, rising to **98.97%** for AW=128.

End-to-End Speedup. By completely eliminating stall cycles for all MINISA configurations, MINISA achieves geometric mean speedups of $1.87\times$ **FEATHER** (16×16), $6.71\times$ (32×32), $25.5\times$ (64×64), and $99.4\times$ (128×128). This confirms that VN abstraction is essential for scaling reconfigurable accelerators. Without it, control overhead becomes dominating bottleneck for large arrays.

2) Performance Evaluation of Modified Architecture: We further analyze the architectural efficiency of FEATHER+ under MINISA by breaking down the execution cycles of representative workloads in Fig. 10. The breakdown explicitly tracks the latency of concurrent activities: exclusive computation (Compute), input/weight streaming (Load In/W), output draining (Out→Stream), and off-chip transfers (Store Out).

Hiding Data Movement Overheads. A key architectural advantage shown in Fig. 10 is the effective masking of memory access latencies. Thanks to the double-buffered (ping-pong) design of the Streaming and Stationary buffers, data movement phases—loading inputs/weights and draining outputs—occur in parallel with the Compute phase. By decoupling the memory triggers (via SetLayout instructions) from the execution triggers (SetMapping), MINISA ensures that the execution pipeline is rarely stalled by data marshalling. The explicit Load Inst overhead is rendered negligible, validating that the coarse-grained instruction stream does not compete for bandwidth or incur fetch stalls.

Robustness to Irregular Shapes. The most significant insight from Fig. 10 is the stability of Compute Utilization (red line) across diverse tensor shapes.

• *Irregular Workloads:* For FHE and ZKP kernels with highly irregular dimensions (e.g., left-side cases where $K = 10$ or $N = 21$) , FEATHER+ maintains a compute utilization of $> 98\%$. This sharply contrasts with rigid systolic arrays, which typically suffer severe utilization drops due to padding when dimensions do not align with array sizes.

• *Regular Workloads:* For standard power-of-two shapes (e.g., right-side cases with $K, N \in \{1024, 2048\}$) , utilization approaches ideal peaks ($> 99.4\%$).

This confirms that the Virtual Neuron (VN) abstraction allows FEATHER+ to map both "odd" and regular shapes with equal efficiency, utilizing available PEs without the fragmentation penalties typical of fixed-function accelerators.

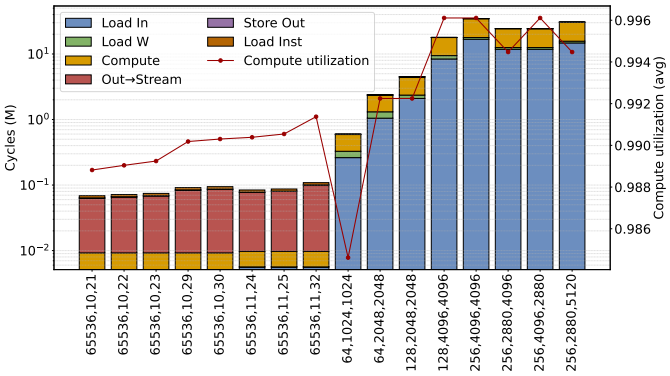

Fig. 10: Latency breakdown and compute utilization of representative workloads. **Takeaway**: FEATHER+ with MINISA could successfully utilize available PEs for both irregular workload shapes, such as K=10 and $K = 2^n, n \in \mathbb{Z}^+$.

D. Resource Overhead Evaluation (Q3)

Finally, we assess the hardware cost of the architectural refinements in FEATHER+ (all-to-all distribution) required to support MINISA. Tab. VI summarizes the area comparison.

Area Scalability. The area overhead of FEATHER+ over FEATHER decreases as the array size increases. For small arrays (4×4), the overhead is 8.37% due to the fixed cost of control logic relative to a small compute area. However, for practical large-scale arrays, the overhead becomes negligible: **0.62%** for 64×64 and **0.75%** for 64×128 configurations. This confirms that FEATHER+ enables dynamic VN-level programmability with minimal silicon cost.

TABLE VI: Area (μm^2) evaluation (TSMC 28nm). **Takeaway:** FEATHER+ adds negligible resources to FEATHER

Setup	FEATHER	FEATHER+	Overhead
4×4	24694	26950	8.37%
8×8	97976	105376	7.02%
16×16	475897	493612	3.59%
32×32	2727907	2774313	1.67%
64×64	18389176	18504713	0.62%
64×128	36920520	37198574	0.75%

In summary, our evaluation demonstrates that MINISA solves the scalability crisis of reconfigurable accelerators. By shifting from wire-level micro-control to VN-level abstraction, MINISA reduces control traffic by up to 5 orders of magnitude and unlocks nearly $100\times$ speedups at scale, all while incurring less than 1% area overhead for large arrays.

VI. Related Work

Instruction Set for AI Accelerators. Commercial AI accelerators typically employ monolithic instruction sets driving fixed functional units to maximize efficiency for standard deep learning operators. Architectures like Google's TPU [9], Meta's MTIA [6], and Amazon's Trainium [2] rely on VLIW or CISC-like abstractions that assume regular, dense computation. While efficient for standard CNNs and Transformers, these rigid ISAs lack the fine-grained distinct programmability required to support the irregular dataflows and complex algebraic structures found in emerging workloads like Homomorphic Encryption (HE) and Zero-Knowledge Proofs (ZKP).

Reconfigurable Dataflow Accelerators. To address the diversity of tensor shapes and operators, reconfigurable architectures such as MAERI [16], SIGMA [22], and Flexagon [20] introduce flexible interconnects to support diverse dataflows [21]. Further efforts like PolyGraph [4], DSA-Gen [31], and Over-Gen [18] explore overlay generation to map varied kernels. However, these designs often decouple computation from memory layout, incurring significant data marshalling overheads. FEATHER [29] addresses this by enabling low-cost co-switching of dataflow and data layout. *Crucially, however, prior reconfigurable baselines—including FEATHER—rely on fine-grained micro-configuration.* As array sizes scale, this results in excessive control overhead and instruction-fetch stalls that bottleneck end-to-end performance. MINISA specifically targets this scalability crisis by raising the abstraction level to Virtual Neurons, eliminating redundant control traffic while preserving architectural flexibility.

Hardware Acceleration for Emerging Cryptography (HE & ZKP). The high computational cost of cryptographic primitives has driven a bifurcation in hardware support. *Software and FPGA approaches*, such as Over100x [12], TensorFHE [5], HEAX [23], and FAB [1], offer programmability but are often bound by memory bandwidth or clock frequency. *Dedicated ASICs*, including F1 [24], CraterLake [25], BASALISC [7], BTS [13], and recent ZKP accelerators like UniZK [30], achieve peak efficiency but are fixed-function devices incapable of adapting to evolving algorithms or general AI workloads. MINISA bridges this gap by enabling a general-purpose reconfigurable substrate (FEATHER+) to efficiently support the complex NTT and ring-learning-with-errors operations inherent to HE and ZKP [28], delivering ASIC-approaching performance without the cost of single-domain silicon.

VII. Conclusion

This paper presents MINISA, the first Instruction Set Architecture to enable simultaneous dataflow and layout reconfigurability for Deep Learning and emerging cryptographic workloads. Our central insight is the Virtual Neuron (VN) abstraction, the atomic hardware dot-product unit, which identifies the precise granularity required to decouple algorithmic mapping from physical execution. By pairing this abstraction with the FEATHER+ architecture, we successfully extend general-purpose acceleration to the irregular algebraic structures found in FHE and ZKP (e.g., NTT) without incurring the bandwidth penalties of micro-coded baselines.

The empirical impact is decisive. MINISA resolves the scalability crisis of reconfigurable hardware, compressing off-chip control traffic by a geometric mean of $1.9\times10^5\times$ and completely eliminating instruction-fetch stalls that otherwise consume up to 98% of cycles. This efficiency translates to $99.4\times$ end-to-end speedups on large-scale arrays (128×128), proving that extreme programmability can be achieved with negligible control overhead.

REFERENCES

[1] R. Agrawal, L. de Castro, G. Yang, C. Juvekar, R. Yazicigil, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, "Fab: An fpga-based accelerator for bootstrappable fully homomorphic encryption," 2022.

[2] AWS. (2025) Aws nki instruction set architecture. [Online]. Available: https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/api/nki.isa.html

[3] A. A. Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, I. Quah, Y. Polyakov, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca, "Openfhe: Open-source fully homomorphic encryption library," Cryptology ePrint Archive, Paper 2022/915, 2022, https://eprint.iacr.org/2022/915. [Online]. Available: https://eprint.iacr.org/2022/915

[4] V. Dadu, S. Liu, and T. Nowatzki, "Polygraph: exposing the value of flexibility for graph processing accelerators," in *Proceedings of the 48th Annual International Symposium on Computer Architecture*, ser. ISCA '21. IEEE Press, 2021, p. 595–608. [Online]. Available: https://doi.org/10.1109/ISCA52012.2021.00053

[5] S. Fan, Z. Wang, W. Xu, R. Hou, D. Meng, and M. Zhang, "Tensorfhe: Achieving practical computation on encrypted data using gpgpu," 2022.

[6] A. Firoozshahian, J. Coburn, R. Levenstein, R. Nattoji, A. Kamath, O. Wu, G. Grewal, H. Aepala, B. Jakka, B. Dreyer, A. Hutchin, U. Diril, K. Nair, E. K. Aredestani, M. Schatz, Y. Hao, R. Komuravelli, K. Ho, S. Abu Asal, J. Shajrawi, K. Quinn, N. Sreedhara, P. Kansal, W. Wei, D. Jayaraman, L. Cheng, P. Chopda, E. Wang, A. Bikumandla, A. Karthik Sengottuvel, K. Thottempudi, A. Narasimha, B. Dodds, C. Gao, J. Zhang, M. Al-Sanabani, A. Zehtabioskuie, J. Fix, H. Yu, R. Li, K. Gondkar, J. Montgomery, M. Tsai, S. Dwarakapuram, S. Desai, N. Avidan, P. Ramani, K. Narayanan, A. Mathews, S. Gopal, M. Naumov, V. Rao, K. Noru, H. Reddy, P. Venkatapuram, and A. Bjorlin, "Mtia: First generation silicon targeting meta's recommendation systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3579371.3589348

[7] R. Geelen, M. V. Beirendonck, H. V. L. Pereira, B. Huffman, T. McAuley, B. Selfridge, D. Wagner, G. Dimou, I. Verbauwhede, F. Vercauteren, and D. W. Archer, "BASALISC: Programmable hardware accelerator for BGV fully homomorphic encryption," Cryptology ePrint Archive, Paper 2022/657, 2022. [Online]. Available: https://eprint.iacr.org/2022/657

[8] M. Gilbert, Y. N. Wu, A. Parashar, V. Sze, and J. S. Emer, "Looptree: Enabling exploration of fused-layer dataflow accelerators," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023, pp. 316–318.

[9] N. P. Jouppi, D. Hyun Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. Patterson, "Ten lessons from three generations shaped google's tpuv4i : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1–14.

[10] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. Patterson, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," 2023.

[11] N. P. Jouppi, D. H. Yoon, G. Kurian, S. Li, N. Patil, J. Laudon, C. Young, and D. A. Patterson, "A domain-specific supercomputer for training deep neural networks," *Commun. ACM*, vol. 63, no. 7, pp. 67–78, 2020. [Online]. Available: https://doi.org/10.1145/3360307

[12] W. Jung, S. Kim, J. H. Ahn, J. H. Cheon, and Y. Lee, "Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with gpus," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 114–148, 2021.

[13] S. Kim, J. Kim, M. J. Kim, W. Jung, J. Kim, M. Rhu, and J. H. Ahn, "Bts: An accelerator for bootstrappable fully homomorphic encryption," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 711–725. [Online]. Available: https://doi.org/10.1145/3470496.3527415

[14] T. Krishna, H. Kwon, A. Parashar, M. Pellauer, and A. Samajdar, "Data orchestration in deep learning accelerators," 2020.

[15] H. Kwon, P. Chatarasi, V. Sarkar, T. Krishna, M. Pellauer, and A. Parashar, "Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings," *IEEE Micro*, vol. 40, no. 3, pp. 20–29, 2020.

[16] H. Kwon, A. Samajdar, and T. Krishna, "MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnects," in *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2018.

[17] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Comput. Surv.*, vol. 52, no. 6, oct 2019. [Online]. Available: https://doi.org/10.1145/3357375

[18] S. Liu, J. Weng, D. Kupsh, A. Sohrabizadeh, Z. Wang, L. Guo, J. Liu, M. Zhulin, R. Mani, L. Zhang, J. Cong, and T. Nowatzki, "Overgen: Improving fpga usability through domain-specific overlay generation," in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '22. IEEE Press, 2023, p. 35–56. [Online]. Available: https://doi.org/10.1109/MICRO56248.2022.00018

[19] W. Ma, Q. Xiong, X. Shi, X. Ma, H. Jin, H. Kuang, M. Gao, Y. Zhang, H. Shen, and W. Hu, "Gzkp: A gpu accelerated zero-knowledge proof system," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 340–353. [Online]. Available: https://doi.org/10.1145/3575693.3575711

[20] F. Muñoz-Martínez, R. Garg, J. L. Abellán, M. Pellauer, M. E. Acacio, and T. Krishna, "Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient dnn processing," 2023. [Online]. Available: https://arxiv.org/abs/2301.10852

[21] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A Systematic Approach to DNN Accelerator Evaluation," in *Proceedings of the International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019.

[22] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 58–70.

[23] M. S. Riazi, K. Laine, B. Pelton, and W. Dai, "Heax: An architecture for computing on encrypted data," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1295–1309. [Online]. Available: https://doi.org/10.1145/3373376.3378523

[24] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, "F1: A fast and programmable accelerator for fully homomorphic encryption," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 238–252. [Online]. Available: https://doi.org/10.1145/3466752.3480070

[25] N. Samardzic, A. Feldmann, A. Krastev, N. Manohar, N. Genise, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, "Craterlake: A hardware accelerator for efficient unbounded computation on encrypted data," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 173–187. [Online]. Available: https://doi.org/10.1145/3470496.3527393

[26] J. Seo, J. Tong, T. Krishna, and H. Kwon, "Exploring constrained dataflow accelerators for real-time multi-task multi-model ml workloads," in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2025, pp. 1–11.

[27] J. Strzeszynski, J. Tong, K. Lee, N. Xiong, A. Parashar, J. S. Emer, T. Krishna, and M. Yan, "Squareloop: Explore optimal authentication block strategy for ml," in *Proceedings of the 14th International Workshop on Hardware and Architectural Support for Security and Privacy*, ser. HASP '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 37–45. [Online]. Available: https://doi.org/10.1145/3768725.3768732

[28] J. Tong, T. Huang, L. de Castro, A. Itagi, J. Dang, A. Golder, A. Ali, J. Jiang, Arvind, G. E. Suh, and T. Krishna, "Leveraging

asic ai chips for homomorphic encryption," 2025. [Online]. Available: https://arxiv.org/abs/2501.07047

[29] J. Tong, A. Itagi, P. Chatarasi, and T. Krishna, "Feather: A reconfigurable accelerator with data reordering support for low-cost on-chip dataflow switching," in *Proceedings of the 51th Annual International Symposium on Computer Architecture*, ser. ISCA '24. Argentina: Association for Computing Machinery, 2024.

[30] C. Wang and M. Gao, "Unizk: Accelerating zero-knowledge proof with unified hardware and flexible kernel mapping," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1101–1117. [Online]. Available: https://doi.org/10.1145/3669940.3707228

[31] J. Weng, S. Liu, V. Dadu, Z. Wang, P. Shah, and T. Nowatzki, "Dsagen: Synthesizing programmable spatial accelerators," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 268–281.